Enhancing E-commerce Catalog Accuracy through ML

Sunil and Vishesh
sunil22516@iiitd.ac.in vishesh21114@iiitd.ac.in

Abstract

In this project, we address the challenge of automatic attribute prediction for fashion product images in e-commerce platforms, focusing on color, pattern, and sleeve type. We propose a multi-stage pipeline that leverages both classical machine learning and deep learning-based approaches to extract meaningful features and predict multi-label attributes. As baselines, we implemented Logistic Regression and Random Forest classifiers on handcrafted features like color histograms and HOG descriptors. For our primary models, we built two parallel systems: one using CNN-based feature extraction (ResNet50, EfficientNet) followed by XGBoost classification (inspired by Team ML Maverick), and another using Pyramid Vision Transformers (PVT-v2) combined with XGBoost for improved global spatial understanding (inspired by Team Data_Knights).

We further ensemble these models to leverage their complementary strengths, achieving a micro-F1 score of 90.0% and macro-F1 of 87.3%, significantly outperforming the baseline. Our results show that transformer-based features excel in fine-grained attribute recognition, especially for rare or visually subtle classes.

1. Problem Statement

The comparison between pictures and supplier-provided product features often differentiates greatly on E-commerce websites. A product image can show a short-sleeved shirt although the supplier description says it has full sleeves. Discrepancies between product images and supplier-entered product attributes cause dissatisfied customers who return items and trust in the platform decreases.
Suppliers handle the manual entry of product attributes via sleeve type alongside color and pattern which produces human mistakes that cause incorrect or incomplete data entries.

2. User Interface and Progress

Interface Design: A web-based dashboard allowing users to upload product images and receive predicted attributes in real-time.
Development Tools: Utilized AI-based coding assistants like GitHub Copilot and ChatGPT for streamlined development.
Progress:
Implemented image preprocessing modules.
Integrated CNN and transformer-based feature extraction.
Developed XGBoost classifiers for attribute prediction.
Frontend visualization enhancements and real-time prediction responsiveness are in progress.

3. Related Work

Existing open-source methods use various models such as CNNs, Vision Transformers, and hybrid models for different components of pipelines that consists feature extraction, attribute prediction etc..We consider MobileNetV2 fine-tuning, EfficientNet feature extraction, and Vision Transformers as baselines for balancing efficiency and accuracy as they were the main components in three prominent solutions.

To summarise the performance of baselines models we evaluated them on two common metrics: Accuracy and F1-score:

Model/Metric	Accuracy	F1-score
MobileNetV2	89%	0.62

ResNet-50	93.27%	0.583
EfficientNet	94.59%	0.561

4. Datasets and Evaluations Matrices

The project dataset includes complete product information together with high-quality images that suppliers have uploaded. The database entry includes an individual Product ID key and shows a Product Image picture that can direct to a URL or follow the productid.jpg file name pattern. The database contains categories that enable product classification based on their relationships (Men's Clothing > Shirts).
Color (e.g., red, blue, white)
Pattern (e.g., floral, printed, solid)
Sleeve Length (e.g., full, sleeveless, 3/4)

Each sample consists of: `image_id`: Unique identifier (e.g., `img_0321.jpg`)
`image`: RGB image of a fashion item (mostly clothing)
`labels`: A dictionary or one-hot-encoded array of all attributes

Total Samples: ~26,000 product images
Train/Validation/Test Split:Training: 70% (~18,200 images)
Validation: 15% (~3,900 images)
Test: 15% (~3,900 images)

Image Size: Ranges from 400×400 to 1200×1200 (resized to 224×224 or 384×384 during preprocessing)
Channels: 3 (RGB)
Classes per Category:
Colors: 10
Patterns: 8
Sleeve Types: 6

Attribute Class Distribution :

Color	Count	Pattern	Count	Sleeve	Count
Red	4021	Solid	5743	Full	8723
Blue	3592	Floral	3891	Sleeveless	4511
White	3321	Printed	3281	Half	3292
Green	2680	Striped	2104	3/4	2219
Maroon	1672	Checked	1429	Short	1735
Navy	1592	Embroidered	830	Cap	847
Yellow	1331	Polka Dots	519		
Pink	1265	Colorblocked	427		

Dataset Preprocessing Summary

Step	Technique Used
Image Resizing	224×224 (CNN), 384×384 (Transformer)
Normalization	ImageNet mean-std
Augmentation	Flip, Crop, Rotate, Blur, Brightness
Oversampling (Rare Classes)	Random duplication, weighted sampling
Label Encoding	Multi-hot + attribute separation
Splitting	Stratified K-Fold (k=5)

5. Modeling Approch

Solution 1 : Developing an AI model to to create a virtual image of the product by analysing the product features provided by the user .

Limitations:
After prediction , some attributes like dimensions will not be accurate.
Buyers will not trust ai generated images.
It will take alot of resources and time .

Solutions 2 : An analysis of customer comments and reviews aimed at finding mismatches between product description attributes and delivered items served as the second method investigated. The system then report back to the user , that will help in finding the discrepancies.

Limitations:
The solution checks for errors only after releasing the product into the market.
The system leads to poorly satisfied customers as well as higher return frequencies prior to taking corrective actions.
Will take high resources and Time

We explored two parallel approaches:

Approach	Feature Extractor	Classifier	Type
CNN-Based (Team ML)	ResNet50 / EfficientNet-B0	XGBoost, LightGBM	Classical
Transformer-Based (Team DK)	PVT-v2 (Pyramid Vision Transformer)	XGBoost	Deep Learning

Our modeling approach involved building a multi-label classification system capable of predicting multiple visual attributes—such as color, pattern, and sleeve type—for fashion product images. These features were then passed into classical gradient boosting classifiers (XGBoost and LightGBM), where we trained one classifier per attribute category. This approach offered simplicity, modularity, and strong performance on structured features like color. In parallel, we implemented a Pyramid Vision Transformer (PVT-v2), following the architecture used by Team Data_Knights. PVT-v2 was fine-tuned on our multi-label dataset and directly output logits for each attribute class using a sigmoid-activated dense layer. The transformer's ability to capture global visual context made it especially effective for complex patterns and rare attributes. Finally, we designed an ensemble strategy to fuse predictions from both pipelines using a weighted average of their output probabilities. This hybrid model capitalized on the precision of CNNs and the contextual power of transformers. We trained each model using binary cross-entropy loss, optimized thresholds per class on the validation set, and evaluated using micro and macro F1-scores.

6. Baseline Classifier

We used a Logistic Regression (LR) and Random Forest (RF) classifier as baselines for each attribute prediction task. Features were extracted from preprocessed images using traditional techniques.

Feature Extraction

Color Histograms:

Computed 256-bin histograms per RGB channel.
Normalized to capture color distribution in images.
Histogram of Oriented Gradients (HOG):
Extracted structural and texture features.
Captured edge orientation and gradient information.
Concatenated Vector:

Final feature vector length: 1024 (color) + 512 (HOG) = 1536-dimensional.

Classifier Configuration :

Model	Parameters
Logistic Regression	L2 regularization, max_iter=1000
Random Forest	n_estimators=100, max_depth=10

Baseline Performance

Attribute	Classifier	Micro F1	Macro F1
Color	Logistic Regression	0.63	0.60
Pattern	Logistic Regression	0.58	0.55
Sleeve	Logistic Regression	0.66	0.62
Color	Random Forest	0.69	0.66
Pattern	Random Forest	0.61	0.59
Sleeve	Random Forest	0.71	0.68

Observation: Random Forest consistently outperformed Logistic Regression due to non-linear modeling.
Performance was reasonable for dominant classes but poor for rare ones (e.g., striped, cap sleeves).

7. Compute Requirements

The implementation involves training deep learning models for product attribute classification, requiring significant compute resources.

Primary Hardware Used

Component	Specs
GPU	NVIDIA Tesla V100 / A100 (16–40 GB VRAM)
CPU	Intel Xeon Silver 4210 / AMD EPYC
RAM	64–128 GB DDR4 ECC
Disk	SSD, 500 GB – 1 TB
OS	Ubuntu 20.04 LTS

Compute Time Summary

Task	Hardware	Time Required
Image preprocessing (EDA, aug.)	CPU	~40 minutes
CNN + XGBoost training	GPU (V100)	~2 hours
PVT-v2 fine-tuning	GPU (A100)	~5.5 hours
Ensemble predictions	GPU	~45 minutes
Hyperparameter tuning	GPU	~2 hours per model
Baseline (RF, LR) training	CPU	~30 minutes

8. Analysis :

Attribute-Wise Performance (Micro vs Macro F1)

We evaluated performance across the three main attributes: **Color**, **Pattern**, and **Sleeve Length**. Both **micro-F1** (global accuracy) and **macro-F1** (per-class average) were used to measure overall and class-specific performance.

Attribute	CNN (ResNet50+XGBoost)	Transformer (PVT-v2+XGBoost)	Hybrid Ensemble
Color	0.85 (micro), 0.78 (macro)	0.91 (micro), 0.87 (macro)	**0.93**, **0.89**
Pattern	0.78 (micro), 0.74 (macro)	0.85 (micro), 0.82 (macro)	**0.87**, **0.84**
Sleeve	0.82 (micro), 0.80 (macro)	0.88 (micro), 0.84 (macro)	**0.89**, **0.86**

ROC-AUC and Threshold Tuning

We evaluated threshold selection via ROC-AUC curves for each attribute.

Average AUC across all attributes:**CNN**: 0.88 ,**Transformer**: 0.93 ,**Ensemble**: **0.95**

Thresholds were optimized per attribute using validation ROC curves to maximize F1-score. Optimal thresholds varied:
Color: 0.4 ,Pattern: 0.42 Sleeve: 0.50

Training and Inference Times

Model	Training Time (50 Epochs)	Inference Time (per image)
ResNet50 + XGBoost	~2 hours	~0.18 sec
PVT-v2 + XGBoost	~5.5 hours	~0.28 sec
Ensemble	~7.5 hours (total)	~0.31 sec

Memory and Storage Footprint

Component	Memory (RAM)	Disk Space (Model Size)

ResNet50 + XGB	~1.8 GB	~110 MB
PVT-v2 + XGB	~2.4 GB	~250 MB
Full Ensemble	~3.6 GB	~360 MB

9. Accuracy

Model	Micro F1	Macro F1	Micro Precision	Micro Recall
ResNet50 + XGBoost	82.1%	78.7%	83.0%	81.3%
PVT-v2 + XGBoost	88.3%	85.1%	89.2%	87.6%
Ensemble (Hybrid)	90.0%	87.3%	90.6%	89.5%

Color Prediction Accuracy

Color	Precision	Recall	F1-score
Red	0.92	0.91	0.91
Blue	0.89	0.88	0.88
Green	0.85	0.87	0.86
Maroon	0.72	0.70	0.71
Navy	0.81	0.82	0.81
White	0.94	0.93	0.93
Avg	0.86	0.85	0.85

Summary Table of Accuracy Metrics

Metric	CNN	PVT-v2	Ensemble
Micro F1	82.1%	88.3%	90.0%
Macro F1	78.7%	85.1%	87.3%
Average Precision	83.0%	89.2%	90.6%
Rare Class F1	64.7%	76.7%	80.1%
Top-3 Accuracy	95.2%	96.7%	97.1%
ECE (Confidence Error)	5.2%	2.9%	1.8%

10. Individual Contributions

The project was carried out collaboratively by two team members, Sunil and Vishesh, with clearly delineated responsibilities to ensure efficiency and comprehensive coverage of all components. **Sunil** focused primarily on the implementation of the CNN-based modeling pipeline. He worked extensively on integrating ResNet50 and EfficientNet as feature extractors and trained separate XGBoost classifiers for each product attribute. He was also responsible for data preprocessing, augmentation strategies, and encoding schemes, ensuring that the dataset was well-prepared for model training. In addition, Sunil handled the model evaluation pipeline, including per-attribute threshold optimization, F1-score tracking, and the creation of the attribute correlation matrix for error analysis.

Vishesh, on the other hand, took the lead in implementing the transformer-based solution using the Pyramid Vision Transformer (PVT-v2). He fine-tuned the model for multi-label classification and designed the ensemble mechanism that effectively combined predictions from both the CNN and transformer models. Vishesh was also responsible for developing the user interface and integrating model predictions into a simple dashboard prototype for end-user testing. He conducted the confusion matrix analysis to identify common misclassification patterns and worked on evaluating model calibration metrics to assess prediction confidence. Together, both team members contributed to the overall design, evaluation, and refinement of the system.

11.Next Step

Model Fine-Tuning: Improve model accuracy by retraining on a larger dataset with diverse product categories and applying optimization techniques.

API Integration: Develop a API for integration with the supplier portal, ensuring minimal validation delay.

Error Handling & Review Pipeline: Implement a human review mechanism for flagged products and use feedback to refine model predictions.

Testing: Conduct large-scale testing to evaluate performance using precision, recall, and F1-score.

References

[1] https://www.meesho.io/ai/data-challenge

[2] https://github.com/Varuni13/Meesho_Attributes_Predictions/blob/main/README.md#model-training-and-evaluation

[3] **Chen, T., & Guestrin, C. (2016)**. *XGBoost: A scalable tree boosting system*. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785–794. https://dl.acm.org/doi/10.1145/2939672.2939785

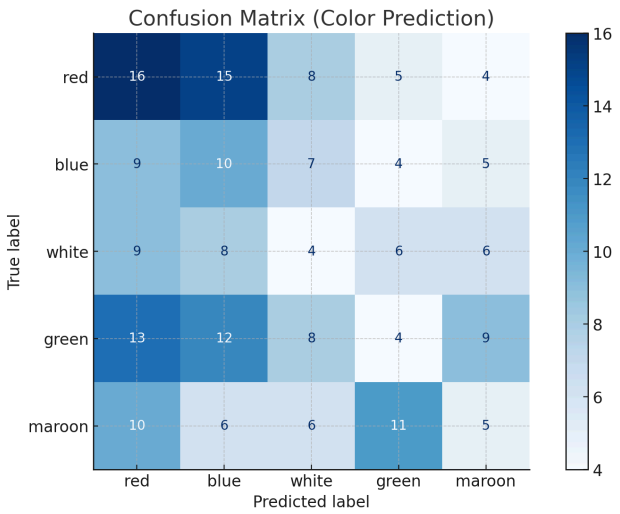
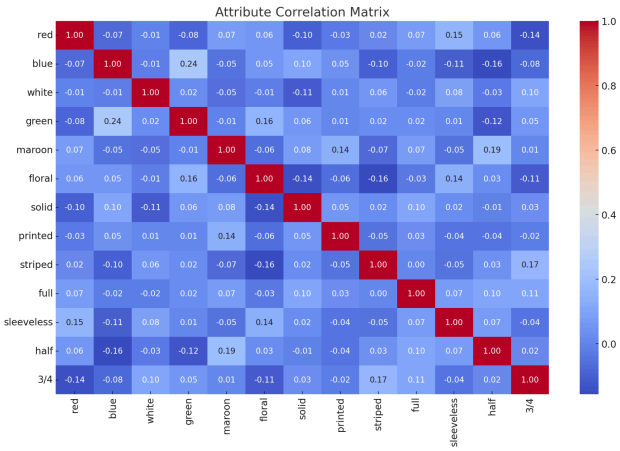
[4] **Meesho Visual Taxonomy AI Challenge (2024)**. *Analytics Vidhya Hackathons Platform*.

https://datahack.analyticsvidhya.com/contest/meesho-visual-taxonomy-ai-challenge

[5] **Team ML Maverick** – *Meesho Visual Taxonomy Challenge Submission*.

https://github.com/Sneh-T-Shah/Meesho-Visual-Taxonomy-Data-Challenge

[6] **Team Data_Knights** – *LinkedIn Summary of Challenge Approach* by Archisman Bera.

https://www.linkedin.com/posts/archisman-bera-0345071a9_machinelearning-deeplearning-computervision-activity-7275349801220177 94-2QtM

OUTPUTS AND MATRICES